

集创赛

视频教程

基于紫光同创的 FPGA 智能交通感知系统

【[竞赛·分享]2026 第十届集创赛·芯片应用·FPGA 应用开发·紫光同创杯·基于紫光同创的 FPGA 智能交通感知系统——赛题解析】 https://www.bilibili.com/video/BV1oLfjBpEc8/?share_source=copy_web&vd_source=a118d850a54934837bc1df04aeff815d

黑金 fpga、野火 fpga、正点 fpga

海云捷讯图像识别

2025 年嵌赛 - 基于安路 FPGA 实时图像信号传输系统

正文

Turn 1



这个赛题是基于紫光同创 FPGA 的智能交通感知系统的一道题目，题目简单描述为：

随着城市的迅速发展，城市交通环境也变得更加复杂，行人行为及车牌识别作为智能交通的核心感知技术，在车辆通行管理、交通违章抓拍、ETC 闸机、停车场自动化运维等场景中具有不可替代的作用。传统基于 CPU / GPU 的行人和车牌识别方案存在实时性不足、功耗较高、部署灵活性受限等问题，而 FPGA + MCU 凭借并行计算架构、低延迟、高性价比的优势，成为嵌入式场景下智能交通感知系统的理想硬件平台。

要求基于紫光同创 FPGA 芯片，采用纯逻辑方案或移植 RISC - V 的软核（提供 RISC - V 参考），或采用 ARM + 紫光同创 FPGA 方案（ARM 的处理性能不应超过 RK3568），完成包含图像采集接口（HDMI 或者摄像头输入）、预处理模块、车牌识别、结果输出的硬件架构；

关键算法实现：

- (1) 图像预处理：基于 FPGA 实现图像灰度化、降噪（如中值滤波、高斯滤波）、二值化等算法，解决复杂光照（强光、阴影）、模糊等场景下的图像质量问题；
- (2) 车牌定位与识别：通过边缘检测、形态学操作、颜色分割等算法，在 FPGA 中并行实现车牌区域快速提取，或者基于 AI 算法模型进行车牌的识别和定位，支持蓝牌、绿牌等常见车牌和特殊车牌；
- (3) 车牌定义：本文车牌特指机动车号牌，其相关标准参考《中华人民共和国机动车号牌》(GA36-2018) 标准。其蓝牌和绿牌分别为小型汽车号牌和绿牌指小型新能源车号牌，在本文中认定为常见车牌；大黄牌（大型汽车号牌）、小黄牌（摩托车号牌）、警用号牌、挂车号牌、领馆号牌、使馆号牌等类型认定为特殊车牌。

系统集成验证：

- (1) 搭建硬件实验平台：连接 FPGA 开发板与图像采集模块（HDMI、摄像头、以太网的一种或几种）、显示模块（LCD / HDMI）、存储模块（SD 卡），实现图像采集 - 处理 - 识别 - 结果输出的全流程闭环；
- (2) 软件辅助设计：使用 Verilog / VHDL 语言编写硬件逻辑代码，或搭配 RISC - V 嵌入式软核（或 ARM）实现系统控制与数据交互；
- (3) 功能与性能测试：
- A. 在不同场景（晴天、阴天、夜间、远距离、倾斜角度）下采集测试样本，验证系统识别准确率、识别延迟、实时帧率等指标；
- B. 支持特殊车牌识别；
- C. 低功耗优化设计，适配嵌入式移动场景

评分标准

基础要求：20 分 1、完成视频的采集输入、识别、输出整个链路（20 分）

提高要求：60 分

1、对车牌进行识别，并对车牌进行标记，针对不同场景下一个或者多个车牌进行标记，标记准确率越高得分越高：（15 分）

2、不同角度及光线强度下识别准确率，识别率越高得分越高；（满分 15 分）

识别延迟；系统延迟越低，得分越高；（满分 15 分）

特殊车牌识别，一种特殊车牌识别 5 分，2 种特殊车牌识别得 7 分，大于三种车牌识别 10 分；（满分 10 分）

4、创新能力：能完成行人的识别，并对行人违法行为进行标记：行人闯红灯、行人横穿马路、车辆逆行等的一种或几种常见交通违法行为。(5分)

设计完整性：10分

1、完整设计方案(3分)

2、完整仿真报告(3分)

3、完整功耗、时序、面积、性能测试与展示报告(4分)

现场答辩与团队协作

现成答辩(5分)

1、PPT 条例清晰；

2、文档逻辑通顺，图示美观；作

3、源码规范，注释清晰。

团队协作(5)

1、每个团队成员均在项目中发挥重要作用；

2、团队能力互补、协作；

3、现场每个成员需简述自己的工作。

推荐板卡及外设

这里有好多种，在这里先列举其中一种推荐推荐板卡及外设，其他有需要后面再补充

1. RK3568_MES2L50H

RK3568_MES2L50H 核心板采用紫光创 Logos2 系列 FPGA，型号：PG2L50H - 6IFBG484，ARM 采用瑞芯微 RK3568。开发板采用核心板 + 扩展板结构，核心板与扩展板之间使用高速板对板连接器进行连接。核心板的 FPGA 部分主要由 FPGA + 1 颗 DDR3 + 2 颗 FLASH + 电源及复位电路组成，承担了 FPGA 最小系统运行及高速数据处理及存储功能。

FPGA 选用紫光同创 28nm 工艺 FPGA (logos2:PG2L50H - 6IFBG484 / PG2L100H - 6IFBG48);PG2L50H (PG2L100H) 与、DDR3 在数据交互时时钟频率最、高可达 1066Mbps，DDR3 数据位宽为 16bit，因此数据带宽可达 (1066Mbps * 16)，充分满足高速多路数据存储的需求；另外 PG2L50HFPGA (PG2L100H) 带有 4 路 HSST 高速收发器，每路速度高达 6.6Gbps，非常适用于光纤通信与 PCIe 通信；核心板上的 FLASH 主要用于存储 FPGA 配置文件。

RK3568J 是一颗高性能、低功耗的四核应用处理器芯片，它内置了多种功能强大的嵌入式硬件引擎，为端应用提供了优异的性能，RK3568J 为工业级，工作温度范围为 -40°C~85。底板为核心板扩展了丰富的外围接口，其中包括两个光纤模块接口，FMC 接口、40pin 接口，并配置了按键、LED 灯。

我们是决定采用 FPGA + ARM 的方案来实现

请帮我分析题目要求并且列出这个方案下的大纲

Assistant

这份赛题要求实现一个基于紫光同创 **FPGA + ARM (RK3568)** 架构的智能交通感知系统。核心挑战在于**高性能并行计算 (FPGA)** 与**复杂逻辑控制 (ARM)** 的协同，以及对多种车牌（尤其是特殊车牌）和行人违章行为的识别准确率与实时性。

以下是对题目的深入分析及基于 **RK3568 + PG2L50H** 方案的系统设计大纲：

一、题目核心要求分析

1. 硬件架构分工：

- FPGA (PG2L50H)**：负责高带宽、高实时性的任务。包括图像采集 (Sensor/HDMI)、图像预处理 (像素级过滤)、车牌 / 行人定位的加速 (特征提取或卷积加速)。
- ARM (RK3568)**：负责高层逻辑处理。包括系统调度、车牌字符识别 (OCR) 的后端逻辑、特殊车牌语义判断、结果输出 (显示控制 / 以太网传输) 及违章行为的判定。

2. 算法难点：

- 预处理**：需在 FPGA 硬件流水线中完成灰度、降噪、二值化，对不同光照环境有鲁棒性。
- 车牌定位**：需要处理多目标、倾斜角度及不同底色的车牌。
- 特殊车牌识别**：这是提分项 (大黄牌、警牌等)，需要模型或算法具备较强的分类能力。

3. **性能指标**：强调**低延迟**和**高帧率**。由于 RK3568 性能受限 (不超过题目要求)，必须充分发挥 FPGA 的并行加速作用，而非单纯依赖 ARM 运行算法。

二、系统技术方案大纲

1. 硬件平台搭建

- **输入端：**摄像头（MIPI / DVP 接口）或 HDMI 输入。
- **存储端：**FPGA 挂载的 DDR3 用于原始图像缓存（Frame Buffer）；ARM 侧存储识别结果或违章录像。
- **通信链路：**利用 RK3568 与 PG2L50H 之间的高速总线（如 PCIe 或 AXI4-Stream 映射的 FSMC / HPI 接口），实现像素数据与特征数据的高效传输。

2. FPGA 逻辑设计（Verilog / VHDL）

- **视频输入模块：**实现 I2C 配置 Sensor，解析视频同步信号（HSYNC / VSYNC）。
- **图像预处理流水线 (ISP)：**
 - **灰度化 / 色彩空间转换**（RGB to YCbCr）。
 - **空间滤波：**3x3 中值滤波（去除噪点）或高斯滤波。
 - **自适应二值化：**动态调整阈值，应对晴天强光或夜间暗光。
- **特征提取加速器：**
 - **边缘检测：**Sobel 算子定位车牌边缘。
 - **形态学处理：**膨胀与腐蚀，连通域分析（用于初步筛选车牌区域）。
 - **多目标检测算子：**如基于 HOG 或轻量化 CNN 的 FPGA 定点化加速模块。

3. ARM 软件设计（C++/Python/Linux）

- **驱动开发：**编写 FPGA 与 RK3568 的通信驱动（DMA 传输）。
- **车牌识别核心 (LPR)：**
 - **区域裁剪：**接收 FPGA 传回的目标坐标，提取子图。
 - **字符识别：**调用 RK3568 的 NPU 或使用轻量化 OCR 模型（如 HyperLPR 思想）进行字符分割与识别。
 - **标准匹配：**根据 GA36-2018 标准判定车牌类型（蓝、绿、黄、警等）。
- **创新功能：交通违法判定：**
 - 行人检测模型（YOLO-v5/v8-tiny 适配 RK3568 NPU）。
 - 基于 ROI 区域的越界逻辑判断（判定闯红灯、逆行）。

4. 系统集成与优化

- **低功耗设计：**优化 FPGA 资源使用率，利用门控时钟；利用 RK3568 的异构架构平衡能耗。
 - **实时性保障：**采用“双缓冲”或“多缓冲”机制，确保图像采集与处理不掉帧。
-

三、任务分工与开发时间轴（团队参考）

阶段	核心任务	交付物
准备期	硬件连通、RK3568 操作系统裁剪、FPGA 基础点灯 / 采集	硬件 Demo 演示
算法期	FPGA 预处理算法编写、ARM 端模型训练（针对特殊车牌）	仿真波形、模型
集成期	FPGA + ARM 联合调试、数据链路打通、界面开发（HDMI 显示）	完整系统链路
测试期	不同光照 / 角度识别率测试、性能报告（FPS/ 功耗）编写	测试报告、PPT

四、针对评分标准的加分策略

- **关于特殊车牌 (10 分)**: 建议针对 “警”、“使”、“领” 等特殊汉字做特定字符库增强，并在颜色分割算法中增加对黄色、白色的敏感度。
- **关于行人违章 (5 分)**: 这是创新项。可以在 FPGA 侧做运动目标检测（背景建模 / 光流法），在 ARM 侧做目标分类与轨迹判断。
- **关于文档 (10 分)**: 利用紫光同创 PDS 软件导出的**功耗分析报告**和**时序收敛报告**是 “设计完整性” 部分的关键证明。

下一步建议：

您是否已经拿到了 RK3568_MES2L50H 开发板？如果拿到了，我可以为您提供具体的 **FPGA 图像处理流水线（Verilog）** 的通用框架代码 或是 **RK3568 NPU 的模型转换建议**。

Turn 2

User

就按照采集，处理，显示的模块帮我进行划分 (以下只是举例，如果有更好的就采用更好的方案)

采集：OV5640，IIC / SPI

处理：算法（FPGA + ARM）

显示：HDMI/VGA/LCD



Assistant

针对你的硬件平台（RK3568 + PG2L50H），按照“采集 - 处理 - 显示”的链路，我为你梳理了一套成熟且高效的异构设计方案。

这套方案的核心思路是：**FPGA 负责“苦力活”（高带宽像素处理），ARM 负责“脑力活”（逻辑判断与模型推理）。**

1. 系统架构模块划分

采集层 (Data Acquisition)

- 传感器控制**：使用 FPGA 实现 **I2C 控制器**，配置 OV5640 的寄存器（分辨率设为 720P / 1080P，建议 30fps 以上以保证实时性）。
- 数据接口**：OV5640 采用 **DVP 或 MIPI 接口** 输入，FPGA 内部实现数据串解与同步逻辑。
- 多缓冲区管理**：利用 FPGA 侧的 **Video Frame Buffer (VDMA 逻辑)**，将采集到的原始图像写入 DDR3。

处理层 (Processing Pipeline) —— 核心竞争力

这里推荐采用“**预处理 (FPGA) -> 目标定位 (FPGA / ARM) -> 特征识别 (ARM)**”的分工：

模块名称	运行平台	功能描述
ISP 预处理	FPGA	实时完成：灰度化、3x3 中值滤波（去噪）、Sobel 算子（边缘增强）、自适应二值化。
车牌粗定位	FPGA	利用形态学闭运算（Close Operation）进行连通域检测，筛选出疑似车牌的矩形区域。
数据搬运	DMA/PCIe	FPGA 将预处理后的图像或感兴趣区域数据，通过高速接口推送到 RK3568。
AI 推理 (NPU)	ARM	RK3568 调用内置 NPU 运行轻量化模型（如 MobileNet-LPRNet），识别车牌字符。
违法行为判定	ARM	基于多帧追踪算法，判定行人是否闯红灯、车辆是否逆行。

显示层 (Visual Output)

- **OSD 叠加 (On-Screen Display):** FPGA 接收来自 ARM 的坐标信号，在视频流上实时叠加识别框和中英文字符。
- **多路输出:**
 - **HDMI:** 作为主输出，通过 FPGA 内部的 HDMI Encoder（或使用底板集成的芯片）驱动显示器。
 - **LCD / 网口:** RK3568 侧可通过 Qt 界面在小屏幕上显示统计数据。

2. 硬件数据流向图

1. **输入:** OV5640 -> FPGA (DVP/MIPI RX) -> AXI4-Stream。
2. **缓存:** FPGA -> DDR3 Controller (存储原始帧)。
3. **预处理流水线:** DDR3 -> Gray/Filter/Sobel/Morphology -> 疑似区域坐标提取。
4. **跨平台传输:** FPGA -> PCIe/FSMC -> RK3568 (Linux)。
5. **结果回传与显示:** RK3568 将识别出的文本发送给 FPGA OSD 模块 -> HDMI Out。

3. 该方案的优势（对应评分标准）

- **识别延迟 (15 分)**: 由于边缘检测和降噪在 FPGA 硬件级完成，ARM 仅需处理裁剪后的目标图片，极大地降低了端到端延迟。
 - **特殊车牌识别 (10 分)**: 在 RK3568 的 NPU 上部署深度学习模型，比纯 FPGA 逻辑更容易适配黄牌、警牌、领馆牌等复杂特征。
 - **创新能力 (5 分)**: 利用 RK3568 的处理能力，可以并行跑一个行人检测模型，实现行人闯红灯识别。
-

4. 关键技术点建议

- **FPGA 部分**: 建议使用紫光同创的 **PDS (Pango Design Suite)** 调用内置的 DDR3 控制器 IP 和 AXI 总线 IP，确保数据吞吐稳定。
- **ARM 部分**: 由于 RK3568 性能有上限，建议模型采用 **INT8 量化**，以充分压榨其 NPU 算力，保证视频流在 25fps 以上。
- **通信接口**: 如果追求极限速度，首选 PCIe 接口；如果为了开发简便，可以使用 RK3568 的并行总线接口与 FPGA 对接。

下一步建议:

针对“车牌定位”，你是希望采用**传统的形态学方案**（FPGA 纯逻辑实现，速度极快），还是采用**轻量化检测模型**（由 ARM 端的 NPU 完成）？我可以根据你的选择提供具体的逻辑框图。

Turn 3

User

核心板的 FPGA 部分主要由 FPGA + 1 颗 DDR3 + 2 颗 FLASH + 电源及复位电路组成
这个 DDR3 是什么，在这道题目里面具体干什么用的

Assistant

在 FPGA 开发中，**DDR3 (Double Data Rate 3 SDRAM)** 扮演的是“**高速大容量数据仓库**”的角色。

在你的紫光同创 PG2L50H 方案中，FPGA 内部的资源（SRAM / Block RAM）非常有限，通常只有几百 KB 到几 MB，根本无法存下一帧完整的 1080P 高清图像。因此，这颗外部的 DDR3 芯片是实现“智能交通感知”系统的核心支柱。

以下是它在题目场景下的具体作用：

1. 视频帧缓存 (Video Frame Buffer) —— 最主要用途

这是 DDR3 最直观的工作。

- **采集与显示的速率匹配：**OV5640 摄像头采集数据的速度与 HDMI 显示器的刷新频率往往不一致。DDR3 作为一个中间“蓄水池”，FPGA 先将采集到的像素存入 DDR3，然后再由显示模块按需读出。
- **多帧缓存 (Triple Buffering)：**为了防止画面撕裂 (Tearing)，通常会在 DDR3 中开辟 3 个区域，分别用于“正在写入”、“正在处理”和“正在显示”，确保输出图像平滑稳定。

2. 算法处理的“暂存区”

题目要求的图像预处理算法（如中值滤波、形态学操作）需要访问邻域像素：

- **跨行计算：**有些算法需要同时处理相邻的 3 行甚至 5 行数据。虽然 FPGA 内部可以实现行缓存 (Line Buffer)，但如果算法复杂（如大尺寸卷积），就需要借助 DDR3 进行中间结果的读写。
- **背景建模 / 对比：**如果你要做“行人违规”检测，往往需要对比“当前帧”和“背景帧”，DDR3 就是存储这些参考帧的地方。

3. FPGA 与 ARM 的数据中转站 (Shared Memory)

在 RK3568 + FPGA 的架构中，DDR3 往往充当“高速公路服务区”：

- FPGA 采集并预处理完图像（如二值化后），将结果写入 DDR3 的特定地址。
 - ARM 通过总线（如 AXI 或 PCIe）读取该地址的数据，进行后续的车牌字符识别 (OCR) 或违法行为分析。
 - 这样可以避免 ARM 直接去读取慢速的传感器数据，极大地提升了系统整体的**实时性**（15 分）。
-

DDR3 数据流示意图

维度	FPGA 内部寄存器 / BRAM	外部 DDR3 芯片
容量	极小（几千个像素）	很大（通常 256MB 或更多）
速度	极快（单时钟周期访问）	较快（存在访问延迟 / 等待时间）
用途	临时变量、行缓存、FIFO	整帧图像存储、算法中大量数据、多核交互

在本项目中的实际操作建议

在编写 Verilog 代码时，你不需要从零开始写 DDR3 的驱动（那非常复杂）。你需要做的是：

1. **调用 IP 核：**在紫光同创 PDS 软件中调用 **DDR3 Controller IP**。
2. **仲裁逻辑：**编写一个 AXI Interconnect 或简单的仲裁器，因为采集、处理、显示和 ARM 通信都要抢着去读写 DDR3，你需要分配好优先级。

你想深入了解如何通过 AXI 总线来管理这颗 DDR3 吗？我可以为你梳理一个简单的读写仲裁优先级分配方案。

Turn 4

User

我是团队中负责 ARM 板块的人，请问我要从负责 FPGA 的成员那里得到什么，然后得到这个东西之后，我进行什么操作，再把这个结果给到负责 FPGA 的成员

Assistant

作为团队中负责 **ARM (RK3568)** 的成员，你的角色更像是这个系统的“大脑”和“决策中心”。在 FPGA + ARM 的异构架构中，你们之间的数据交互通常遵循“**原始 / 预处理数据上传 -> 高级逻辑处理 -> 结果指令下发**”的闭环。

以下是你与 FPGA 成员协作的详细 SOP（标准作业程序）：

1. 你需要从 FPGA 成员那里得到什么？

FPGA 成员主要为你提供 “半成品” 数据和硬件接口信息。具体包括：

- **数据内容（核心）：**
 - **预处理后的图像数据：**经过 FPGA 灰度化、降噪、甚至二值化后的图像像素（通常存储在 DDR3 的特定地址）。
 - **目标候选框坐标 (ROI)：**如果 FPGA 做了边缘检测或形态学处理，他会给你一组坐标（如：\$x, y, width, height\$），告诉你哪里可能是车牌。
 - **中断信号：**当 FPGA 处理完一帧图像并存入 DDR3 后，会给 ARM 一个硬件中断，通知你 “货到了，快来取”。
 - **开发文档 / 地址表：**
 - **寄存器映射表 (Register Map)：**告诉你通过什么地址可以读取 FPGA 的状态，通过什么地址下发控制指令。
 - **DMA 传输参数：**如果使用 PCIe 或 AXI 直接传输，需要确定数据位宽、首地址和偏移量。
-

2. 得到这些后，你要进行什么操作？（ARM 的核心工作）

你在 RK3568 上通常运行 Linux 系统，你的操作分为 **驱动层** 和 **应用层**：

A. 数据提取（驱动 / 接口层）

- 通过 **Mmap** 或 **DMA 驱动** 访问 FPGA 共享的那块 DDR3 内存。
- 将原始像素数据转换为 OpenCV 或 NPU 模型能够识别的格式（如 RGB / NV12）。

B. 核心算法推理（应用层 - 你的提分点）

- **车牌字符识别 (OCR)：**
 - 使用 RK3568 的 **NPU (RKNN)** 运行轻量化模型。
 - 识别具体的汉字、字母、数字。
- **车牌类型判定：**
 - 根据颜色（蓝 / 绿 / 黄）和字符规则，判定是否为 “特殊车牌”（警牌、使馆牌等）。
- **违法行为逻辑判断：**

- 如果你要做“行人闯红灯”，你需要结合 FPGA 给出的行人位置和当前的交通灯信号（信号量），计算轨迹是否越界。

C. 业务逻辑处理

- 记录识别时间、准确率。
 - 准备好要在屏幕上显示的文本字符串（如：“鲁 A • 88888 - 蓝牌 - 正常”）。
-

3. 你处理完后，要把什么结果给回 FPGA？

为了实现题目要求的“结果输出”和“实时标记”，你需要反馈给 FPGA：

- **OSD (On-Screen Display) 信息：**
 - **文本内容：**将识别出的车牌字符串传回 FPGA，让 FPGA 的 OSD 模块将其渲染在 HDMI 输出画面上。
 - **绘图指令：**告诉 FPGA 在哪个坐标画框（框的颜色可以根据车牌类型变化，比如普通车牌绿框，违章车辆红框）。
 - **控制反馈：**
 - **采集触发 / 停止：**通知 FPGA 调整采集参数（如曝光度、增益），或者切换采集通道。
 - **确认信号：**告诉 FPGA 这一帧已处理完毕，可以释放当前 DDR3 缓冲区供下一帧写入。
-

4. 你们之间的协作流程图

步骤	操作主体	交互内容
1. 采集	FPGA	OV5640 采集 -> 预处理 -> 写入 DDR3 A
2. 通知	FPGA	发送 中断 / 信号 给 RK3568
3. 读取	ARM (你)	通过总线读取缓存区 A 的图像数据
4. 推理	ARM (你)	调用 NPU 识别车牌字符、判定车牌类
5. 反馈	ARM (你)	将“车牌号”和“框坐标”写入 FP 寄存器
6. 显示	FPGA	在 HDMI 实时画面上叠加识别出的文

5. 给你的技术建议（针对 RK3568）

- 利用 NPU 加速：**不要在 ARM CPU 上硬跑识别算法。学习使用 **RKNN-Toolkit**，将训练好的车牌识别模型转换成 `.rknn` 格式，这能极大地降低 **识别延迟（15 分）**。
- 多线程处理：**建立一个生产者 - 消费者模型。驱动层不断拉取 FPGA 数据，识别层并行处理，防止因为 ARM 处理太慢导致 FPGA 侧产生数据积压。
- 定义好协议结构体：**和 FPGA 同学商量好，数据包的头两个字节是什么，校验位是什么，避免因字节序（Endian）问题导致数据错乱。

你想深入了解如何在 RK3568 上调用 NPU 进行车牌识别（OCR）的具体步骤吗？

Turn 5

User

我看有另外一款板卡，描述是这样的

1. 盘古 676 系列开发板

盘古 676 系列开发板（MES2L676，国产 FPGA 权威指南教材的配套板卡），共有 2 款板卡：支持 100k 和 200k 两种逻辑规模，主控分别为：LOg0S-2：PG2L100H - 6IFBG676 和 PG2L200H - 6IFBG676，底板兼容设计。板卡采用了核心板 + 扩展底板的结构，核心板与扩展底板之间使用高速板间连接器进行连接。主控和 DDR3 之间的数据传输时钟频率最高为

533MHz，2 颗 DDR3 的数据位宽为 32bit，总的带宽最高到 34112Mbps (1066x32 = 34112)。另外，MES2L676 板卡带有 8 路 HSSTLP 高速串行收发器，每路的数据传输速率高达 6.6GbpS。MES2L676 开发板预留了 2 路光纤接口、1 路 SMA 高速收发接口、1 路 PCIeGen2x4 数据通信接口、1 路 HDMI 发送接口、1 路 HDMI 接收接口、1 路 10/100/1000 Mbps 的以太网接口，以及 1 组 FMCLPC 扩展接口（符合 FMC 接口规范，可用于外接 FMC 模块）。

这里两颗 DDR3 那位博主是这样介绍的一颗用于图像，一颗用于 AI (权重)，我想知道这代表什么意思？

Assistant

这位博主提到的“一颗用于图像，一颗用于 AI (权重)”是高性能 FPGA 加速设计中非常经典的空间换性能策略。

在盘古 676 开发板上，两颗 DDR3 组合成了 32bit 位宽。博主所说的分工，本质上是在解决**存储带宽瓶颈**。我们可以从以下三个维度来拆解这个逻辑：

1. 为什么要把图像和 AI 权重分开？

在进行“车牌识别”或“行人检测”这种 AI 算法时，FPGA 实际上在同时干两件极其耗费内存带宽的事情：

- **图像流 (动态数据)：**摄像头源源不断地把像素塞进 DDR3，同时预处理模块又要读取这些像素进行灰度化、滤波。这就像是在流水线上处理源源不断的零件。
- **AI 权重 (静态 / 半静态数据)：**神经网络模型（如 YOLO 或轻量化 CNN）由成千上万个参数（权重 \$Weight\$）组成。FPGA 每计算一层卷积，都要从内存中调取大量的权重参数进行乘加运算。

痛点：如果图像和权重挤在同一个 DDR3 控制器下，它们会抢夺总线使用权。当 AI 计算需要加载 10MB 的权重时，图像采集可能就会因为写不进内存而掉帧。**分开存储，相当于给图像和 AI 算法各开了一条专用高速公路。**

2. 具体分工：这代表什么意思？

第一颗 DDR3：图像缓冲区 (Frame Buffer)

- **职责：** 存储 OV5640 采集的原始帧、中值滤波后的处理帧、以及准备交给 HDMI 显示的输出帧。
- **特点：** 数据更新极快（每秒 30-60 帧），读写频繁，对**实时性**要求极高。

第二颗 DDR3：AI 模型 / 权重库 (Weight Storage)

- **职责：** 存储经过训练的车牌识别模型参数（即神经网络的 “大脑记忆”）。
 - **特点：** 数据量巨大，计算时需要短时间内爆发式读取。
 - **在本项目中的意义：** 如果你打算在 FPGA 侧实现部分 AI 加速逻辑（例如车牌区域的粗提取模型），这颗 DDR3 能显著提升计算速度，降低识别延迟。
-

3. 作为 ARM 负责人的你，这代表了什么？

虽然这两颗 DDR3 物理上挂在 FPGA 上，但对你（ARM 侧）的影响很大：

1. **数据交换更清晰：** 通常你会通过 PCIe 或总线访问 FPGA。FPGA 同学可能会告诉你：“图像数据在 A 区（DDR1），识别后的特征向量在 B 区（DDR2）”。你需要写两套不同的地址管理逻辑。
2. **更强的算法上限：**

有了专门存权重的 DDR3，意味着这个系统可以运行**层数更多、更精确**的模型。你不需要为了省内存而过度压缩模型精度，从而提高**识别准确率（15 分）**。

3. 异构协作方案的变化：

- **方案 A（RK3568 方案）：** 你利用 RK3568 内部的 NPU 跑 AI，此时这颗 “AI 权重 DDR3” 可能由你通过总线把模型固件写进去，由 FPGA 读取进行前级加速。
 - **方案 B（纯 FPGA 方案）：** 所有的 AI 运算都在 FPGA 内部完成，此时这两颗 DDR3 会通过 AXI 总线完全由 FPGA 的逻辑代码调度，你只负责接收最终的 “车牌号” 文本。
-

4. 方案对比：你该怎么选？

特性	RK3568 + FPGA (前一个板子)	盘古 676 (纯 FPGA 强项)
开发难度	中 (适合团队协作)。你可以用熟悉的 C++/Python 写逻辑。	高。AI 算法需要用硬件描述语言或 HLS 编写。
内存优势	靠 RK3568 自带的 LPDDR4。	双路 DDR3 (32bit)，带宽极其高 (34Gbps)。
你的任务	重点在 Linux 应用开发 + NPU 模型转换。	重点在 AXI 总线设计 + 硬件实现。

核心建议：

如果你和负责 FPGA 的同学分工明确，且你擅长软件算法，**RK3568 方案**其实更稳健，因为 RK3568 的 NPU 开发生态（RKNN）非常成熟。

如果你想冲击**识别延迟（15 分）**的极限得分，且 FPGA 同学是个“大腿”，那么利用**盘古 676 的双 DDR3 架构**做全硬件加速会更有杀伤力。

既然你负责 ARM，你了解一下在 RK3568 方案中，如何通过 DMA 快速读取 FPGA 第一颗 DDR3 里的图像数据吗？

Turn 6

User

那位博主提到的形态学算法和 AI 算法有什么不同，他说 RK3568_MES2L50H 适合做形态学的方案，而盘古 676 系列开发板比较像网卡和加速卡，本身就是适合设计神经网络加速和 AI 的这种计算单元的设计，为什么

Assistant

这位博主的评价非常专业，他精准地指出了两块板卡在**数据通路设计**和**算力分配**上的本质区别。

作为 ARM 负责人，理解这两者的差异能帮你决定：**你的 AI 模型是跑在 RK3568 的 NPU 上，还是让 FPGA 替你跑一部分。**

1. 形态学算法 vs. AI 算法：本质区别

特性	形态学算法 (Morphology)	AI 算法 (Deep Learning)
原理	基于数学集合论（膨胀、腐蚀、开闭运算）。通过像素点的邻域对比来提取特征。	基于神经网络（卷积、激活、通过数百万次乘加运算（MAC）提取高维特征。
逻辑复杂度	低。主要是加减法、比较器和简单的行缓存（Line Buffer）。	极高。需要海量的浮点 / 定点和巨大的参数存储空间。
灵活性	弱。只能识别形状、颜色、边缘，遇到倾斜或污损车牌极易失效。	强。对光照、角度、复杂背景极高，能识别特殊车牌。
对内存要求	只需要几行像素的缓存。	需要读取巨大的权重文件（Weights）。

2. 为什么 RK3568_MES2L50H 适合 “形态学方案” ？

这款板卡的架构是 “小 FPGA + 强 ARM”。

- **FPGA 规模较小 (50H)**：它的逻辑单元（LUT）和乘法器（DSP）资源有限，很难在内部塞下一个完整的神经网络加速器。
- **硬件分工合理**：FPGA 只做简单的形态学处理（比如：把图像变黑白，把车牌的长方形轮廓勾勒出来），把定位好的 “小图” 扔给 RK3568。
- **RK3568 自带 NPU**：既然 ARM 芯片里已经集成了一个专门跑 AI 的硬核（NPU），你就没必要在 FPGA 里用逻辑电路 “手搓” 一个 AI 单元了。这叫**术业有专攻**。

3. 为什么 盘古 676 适合 “神经网络加速器” 设计？

盘古 676 的定位更接近于 **数据中心级的加速卡**（类似缩小的 NVIDIA 显卡）：

- **资源充沛 (100H / 200H)**：逻辑资源是前者的 2-4 倍，有足够的空间去构建 **PE (Processing Element) 阵列**，也就是所谓的 “AI 计算单元”。
- **双 DDR3 的带宽优势**：
 - **神经网络的痛点是 “访存”**。AI 算法每算一秒钟，可能要读几百 MB 的权重数据。

- 盘古 676 的 32bit 位宽和双 DDR3 设计，允许 FPGA **一边读图像，一边读权重**，互不干扰。这正是“计算单元”设计的核心需求。
 - **接口丰富 (PCIe Gen2x4)**：它预留了高速 PCIe 接口，意味着它可以插在电脑或服务器上，作为一个“外挂脑”来加速复杂计算，这正是“加速卡”的标准形态。
-

4. 对你 (ARM 负责人) 的影响

如果你使用 RK3568_MES2L50H：

- 你的工作重心在 **RKNN 平台**。
- 流程：FPGA 用形态学算法切出车牌区域 -> 通过总线传给你 -> 你调用 RK3568 的 NPU 进行 OCR 识别。
- **优势**：开发周期短，利用现成的 SoC 性能。

如果你使用 盘古 676：

- 你的工作重心可能转向 **驱动开发与协同**。
 - 流程：FPGA 同学在内部写了一个“硬核加速器”，直接把识别出的“字符编码”吐给你。
 - **优势**：系统延迟极低（微秒级），能体现出极强的**底层硬核开发能力**，在赛题的“创新与性能”项（15 分 + 5 分）极具优势。
-

总结建议

- 如果你们追求**快速落地、算法准确**，选 **RK3568 方案**，走“形态学定位 + NPU 识别”路线。
- 如果你们团队有“FPGA 大神”，想挑战**全硬件算子实现**，冲击全国奖项，选 **盘古 676 方案**，尝试在 FPGA 里搭建卷积加速引擎。

既然你负责 ARM，你需要确定一下你的 FPGA 同学是否打算在 FPGA 内部实现卷积算子？这决定了你下一步是去研究 RKNN SDK 还是研究如何写 PCIe / 总线驱动。

Turn 7

我们目前打算走的是 ARM 方案，我承担的角色是负责 RK3568，但是我有点无从下手

Assistant

明白你的顾虑。RK3568 是一个性能非常强劲的 SoC，但也正因为它功能丰富，初学者容易迷失在众多的 SDK 和接口中。

作为 ARM 负责人，你的核心目标只有两个：**数据拿得进来（数据交互）**，**结果算得准确（AI 推理）**。

为了让你不再“无从下手”，我为你整理了一个分阶段的执行路线图，你只需要按部就班地完成以下四个步骤：

第一阶段：搭建开发环境（第 1-3 天）

不要急着写算法，先确保你能控制这块板子。

1. **串口调试**：连接开发板的 Debug 串口，确保能在 PC 上通过终端（如 MobaXterm）进入 Linux 系统。
2. **交叉编译链**：在你的 Ubuntu 电脑上安装 RK3568 的交叉编译工具。你需要编写一个简单的 `Hello World`，编译并在板子上运行成功。
3. **获取 SDK**：向开发板供应商索取 Linux SDK（通常基于 Buildroot 或 Ubuntu 镜像）。

第二阶段：打通 FPGA 通路（核心难点，第 4-10 天）

这是你和 FPGA 成员配合的关键。

1. **确定接口**：询问 FPGA 成员，数据是通过 **内存映射 (mmap)** 还是 **DMA 驱动** 传输。
 2. **读取图像**：尝试从 FPGA 写入的 DDR3 基地址读取一帧数据，并将其保存为 `.bmp` 或 `.jpg` 图片。如果你能看到摄像头拍到的画面，这步就算成功了。
- **提示**：可以使用 Linux 的 `/dev/mem` 设备结合 `mmap` 函数直接操作内存地址。

第三阶段：AI 模型部署（你的主战场，第 11-20 天）

由于你负责 RK3568，利用其 **1TOPS 算力的 NPU** 是得分关键。

1. **选择算法**：去 GitHub 找一个轻量级的车牌识别项目（推荐搜索：`HyperLPR` 或 `YOLOv8-tiny-plate`）。
2. **模型转换 (RKNN)**：

- 在 PC 端使用 `rknn-toolkit` 将训练好的 `.onnx` 模型转换为 `.rknn` 格式。
 - **关键点：**一定要做 **INT8 量化**，这样识别速度会提升 3-5 倍。
3. **编写应用：**使用 C++ 调用 Rockchip 提供的 `rknpu` 库，加载模型，处理从 FPGA 拿到的图像。

第四阶段：业务逻辑与 OSD 反馈（最后阶段）

1. **车牌分类：**写一段简单的逻辑，根据识别出的字符和颜色判定它是“蓝牌”、“绿牌”还是“特殊车牌”（警、使、领）。
2. **结果回传：**按照你们商量好的协议，通过 AXI / 寄存器把识别出的字符串（如 "A88888"）写回 FPGA 对应的地址。
3. **行人违章（创新项）：**如果你还有余力，再部署一个行人检测模型，增加一套简单的“越界判定”数学逻辑。

给你一个具体的“下马威”清单（明天就可以做）：

如果你现在手里有板子，请尝试完成以下动作：

1. **安装 RKNPU 环境：**去瑞芯微官网或 GitHub 下载 `rknpu2` 仓库，跑通它自带的 `rknn_yolov5_demo`。
2. **询问 FPGA 成员：**“兄弟，你打算把采集到的图像存放在 DDR3 的哪个十六进制物理地址？数据是什么格式（RGB888 还是 YUV422）？”

总结：你的工具包

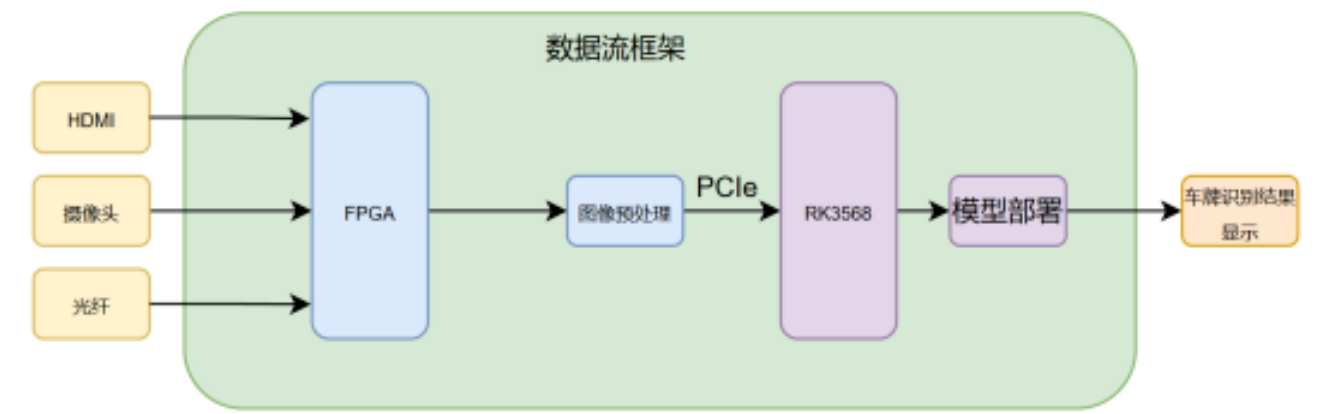
- **语言：**C++ (推荐，因为速度快)
- **库：**OpenCV (图像后处理), RKNPU2 (AI 推理), pthread (多线程)
- **难点：**跨进程读取 FPGA 所在的物理内存地址。

你想让我为你写一个简单的 Linux C++ 代码片段，演示如何通过 `mmap` 读取特定物理地址的图像数据吗？

Turn 8

User

[This turn includes uploaded images]



现在官方出了这个紫光同创赛题的赛题解析，内容如下：

赛题要求

主要考察 FPGA 的图像预处理功能、PCIe 传输、RK 端模型部署优化能力。在 RK3568 有限的算力和资源限制下如何发挥 FPGA 的硬件加速能力，做到软硬件协同处理是重点考察。如图 87b2a74ec732500d36d74eb2c1669ee8.png

推荐设计：

- 1、FPGA 完成 HDMI、摄像头、光纤（任选一种、或者三种都有）的接收。
- 2、FPGA 完成图像预处理（去噪、增强、矫正等）与车牌定位；
- 3、PCIE 的数据传输，RK 端显示
- 4、软硬协同加速处理，FPGA 硬件加速（特别说明了不能够纯靠 RK，也要有 FPGA 的硬件加速不然得分很低），RK 端模型优、提高识别率、帧率，克服特殊场景的识别；

1. 硬件架构与数据流划分

多源视频接入 (FPGA 端)：

√ HDMI: FMC 转 HDMI 子卡实现 1080P 的 HDMI 视频源输入

√ 摄像头: 40PIN 的 IO 扩展双目 OV5640。

√ 光纤: 可以通过另外的 fpga 板卡采集视频流通过光纤传输。

高速数据通信 (FPGA -> RK3568):

√ 建议使用 PCIe 接口进行数据交互。FPGA 将定位并裁剪好的车牌 ROI (感兴趣区域) 图像, 或处理后的全帧图像及坐标, 通过 PCIe DMA 直接写入 RK3568 的内存空间, RK3568 从内存中取出图像数据并处理。

2. FPGA 端: 图像预处理与车牌定位

在 FPGA 上用纯 RTL 实现对应算法

• 图像预处理:

√ 可以实现 RGB 转 HSV, 提取蓝色和绿色的特征掩膜、边缘检测: 车牌区域具有丰富的垂直纹理。使用 Sobel 算子提取垂直边缘。形态学处理: 对边缘图像进行闭运算。

√ 对于倾斜角度还需要 fpga 做矫正处理。强光情况下需要做可以实现基于局部自适应的 Gamma 校正, 对于夜间增强可以采取直方图均衡等, 高阶算法可以考虑使用 clahe 完成。远距离时, 车牌区域实际分辨率降低, FPGA 的识别效果不佳, 可以考虑图像增强、放大车牌区域给到 RK。

3. RK3568 端: AI 识别与分类

RK3568 主要负责对 FPGA 抠出的车牌图像进行语义级别的分析。

模型选择:

车牌分类: 可以通过一个极小的分类网络 (如 MobileNetV2 甚至简单的几层 CNN) 来实现, 或者直接根据 FPGA 端传入的 HSV 色彩判断结果进行综合判定。

字符识别: 推荐使用轻量级的 LPRNet 或 CRNN。不需要复杂的目标检测, 直接输入裁剪好的车牌图像, 输出中文字符和字母数字。

低照度: 可以 RK 部署相关模型实现低照度增强

远距离: 该情况下车牌区域的分辨率相当于降低, RK 可以针对该情况使用轻量级模型去完成识别。



Assistant

根据你提供的官方赛题解析, 比赛的 “风向标” 已经非常明确了。官方强调的是 ** “软硬协同” **, 即不能让 RK3568 一个人干全家的活, 必须把一部分计算压力分给 FPGA。

基于你的 Y9000P 顶配训练环境和即将部署的 RK3568，结合官方推荐设计，你的架构需要做以下重大调整：

1. 任务重新分配：别让 YOLO 跑在 RK3568 上

官方建议 “FPGA 完成图像预处理与车牌定位”。

- **目前的风险：**如果你依然在 RK3568 上跑 YOLOv8，FPGA 就只变成了一个 “采集卡”，这在评分标准里属于 “得分很低” 的纯 RK 方案。
- **优化策略：**
 - **FPGA 端：**利用 Sobel 算子、HSV 色彩过滤（蓝色 / 绿色掩膜）或形态学闭运算来 “圈” 出车牌。
 - **RK3568 端：**只负责跑 LPRNet。
 - **中间桥梁：**FPGA 定位到坐标后，直接裁剪出车牌 ROI（感兴趣区域），通过 **PCIe** 传给 RK3568。

2. 针对特殊场景的 “高分技巧”

官方提到了几个痛点，你可以利用你强大的 i9 笔记本训练针对性的模型：

- **倾斜矫正 (FPGA vs RK)：**

官方建议 FPGA 做矫正。如果 FPGA 实现困难，你可以让 FPGA 传出带有一定余量的车牌图，在 RK 端用 C++ 配合 RGA 硬件进行快速缩放和 Gamma 校正，以克服强光和倾斜。

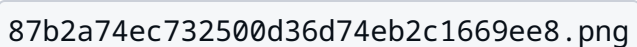
- **远距离 / 低分辨率：**

你的训练集 (CCPD) 里有很多不同距离的样本。针对远距离车牌，你可以在训练 LPRNet 时特意加入一些 ** 模糊处理 (Blur) ** 的样本，提高模型的抗噪能力。

- **低照度增强：**

这是 RK3568 的加分项。如果夜晚识别率低，可以在 RK 端部署一个极小的 **Zero-DCE** 等增强模型，或者直接在 FPGA 端做直方图均衡化。

3. 数据流与部署架构

你的数据流应该参考官方图示 ：

1. **输入：**HDMI / 摄像头 -> FPGA。
2. **定位：**FPGA 利用逻辑资源计算出矩形区域。
3. **传输：**通过 **PCIe DMA** 将车牌小图扔进 RK3568 内存。

4. **识别**：RK3568 触发 NPU 推理 LPRNet，解析字符。

总结建议

虽然你之前的 YOLOv8 训练结果非常漂亮 (mAP 0.99)，但为了符合**紫光同创赛题**的胃口，建议你**将训练重点转向 LPRNet 的极致优化**：

- **训练更轻量的 LPRNet**：让它在 RK3568 上能跑到 100fps 以上。
- **模拟 FPGA 裁剪效果**：在训练 LPRNet 时，不要只给它完美的校正图，故意给一些稍微带点背景、稍微倾斜的图（模拟 FPGA 粗定位的效果），这样你的系统鲁棒性会非常强。

现在的关键点在于 PCIe 传输和 FPGA 的定位算法。你的队友在 FPGA 端实现 HSV 颜色提取和 Sobel 定位有进展吗？如果有，我可以帮你写一个 RK 端的 C++ 接收程序框架。

Exported from [Gemini Voyager](#)

Generated on March 16, 2026 at 08:13 PM

主要是这个 fpga 预处理他得经过

车牌定位：

RGB - RGB 转 YCbCr - (提取 Cr 分量) - 二值化 - 腐蚀 - Sobel 边缘检测 - 膨胀 - 水平投影和竖直投影 - 车牌上下边界和车牌左右边界

字符分割：

RGB -(提取 Red 通道)- 字符区域二值化 - 腐蚀 - 膨胀 - 字符区域水平投影和字符区域竖直投影 - 字符上下边界和字符左右边界

字符识别：

RGB -(提取 Red 通道)- 字符区域二值化 - 腐蚀 - 膨胀 - 提取特征值 - 卷积模板匹配 - 字符识别结果

上面这个是纯 FPGA 实现，现在我们团队是 FPGA 和 ARM 结合的方式，我需要知道就是 FPGA 得要做上面的什么步骤，然后我的 rk 端要做出什么相应的步骤